

UNIVERSIDADE DE MOGI DAS CRUZES
SISTEMAS DE INFORMAÇÃO

CASSIANO JESUS SILVA - 11241104826
GUILHERME DA SILVA BONIFÁCIO - 11241103208
YAN BAUMGARTEN COSTA - 11241100266

RELATÓRIO TÉCNICO DE SEGURANÇA

Esteira DevSecOps com GitHub Actions

Projeto Health In Sight

Instrutor: Fabiano

Mogi das Cruzes, SP

Outubro de 2026

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO	2
2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS	3
2.1 TRUFFLEHOG — SECRETS SCANNING.....	3
2.2 TRIVY — SCA (SOFTWARE COMPOSITION ANALYSIS)	3
2.3 GITHUB CODEQL — SAST (STATIC APPLICATION SECURITY TESTING)	4
3 INTEGRAÇÃO COM O DOCUMENTO DE REQUISITOS	5
4 EVIDÊNCIAS DE FUNCIONAMENTO	6
4.1 ARQUIVO DE PIPELINE (devsecops-pipeline.yml)	6
4.2 EXECUÇÃO DA ESTEIRA — GITHUB ACTIONS	7
4.3 JOB: SECRETS SCANNING (TRUFFLEHOG)	8
4.4 JOB: DEPENDENCY SCAN (TRIVY SCA)	8
4.5 JOB: SAST — CODEQL	8
4.6 POLÍTICA DE BLOQUEIO — COMPORTAMENTO ESPERADO	9
5 CONCLUSÃO	9

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO

O modelo tradicional de desenvolvimento de software trata a segurança como uma etapa final, executada após a conclusão do produto. Essa abordagem gera ciclos longos de correção, custos elevados e exposição contínua a vulnerabilidades. O paradigma DevSecOps (Development + Security + Operations) inverte essa lógica ao integrar controles de segurança diretamente no pipeline de integração contínua — princípio conhecido como Shift Left Security.

Shift Left: mover as verificações de segurança para o início do ciclo de desenvolvimento, interceptando falhas antes que cheguem à produção — reduzindo custo e risco de forma significativa.

Neste projeto, implementamos uma esteira automatizada no GitHub Actions composta por três vertentes complementares de segurança: varredura de segredos expostos, análise de vulnerabilidades em dependências e análise estática do código-fonte. A cada push ou pull request na branch `main`, o pipeline é acionado automaticamente e bloqueia o merge caso encontre falhas de severidade HIGH ou CRITICAL.

2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS

2.1 TRUFFLEHOG — SECRETS SCANNING

O TruffleHog é uma ferramenta open-source especializada em detectar segredos (chaves de API, senhas, tokens, chaves criptográficas) expostos acidentalmente no histórico de commits do repositório Git.

Aspecto	Detalhe
Desenvolvedor	Truffle Security Co.
Método de detecção	Análise de entropia + regex de padrões conhecidos + verificação ativa
Flag utilizada	<code>--only-verified</code> : reporta apenas segredos confirmados como válidos
Integração	GitHub Action oficial: <code>trufflesecurity/trufflehog@main</code>
Disparador	Automático no contexto do push (<code>github.event.before</code> → <code>github.sha</code>)

Fonte: Elaborado pelos autores.

Justificativa: O projeto armazena variáveis sensíveis como `DJANGO_SECRET_KEY`, `FIELD_ENCRYPTION_KEY` (AES-256) e credenciais de banco de dados no arquivo `.env`. Um commit acidental desse arquivo poderia expor dados críticos. O TruffleHog garante que isso seja detectado imediatamente, antes de qualquer merge na branch principal.

2.2 TRIVY — SCA (SOFTWARE COMPOSITION ANALYSIS)

O Trivy, desenvolvido pela Aqua Security, é um scanner de vulnerabilidades abrangente que analisa dependências de projetos (entre outras superfícies) em busca de CVEs catalogados na NVD (National Vulnerability Database) e OSV.

Aspecto	Detalhe
Desenvolvedor	Aqua Security
Modo de varredura	fs — detecta <code>requirements.txt</code> automaticamente
Severidades monitoradas	HIGH e CRITICAL
Política de bloqueio	<code>exit-code: "1"</code> — interrompe o pipeline em falhas críticas
Integração	GitHub Action: <code>aquasecurity/trivy-action@master</code>

Fonte: Elaborado pelos autores.

Dependências do projeto analisadas:

Pacote	Versão	Relevância de segurança
Django	5.2.17	Framework principal — CVEs de XSS, CSRF e bypass de autenticação são comuns
cryptography	50.0.1	Usado para cifrar o segredo 2FA (AES-256) — vulnerabilidades podem comprometer dados em repouso
Pillow	12.3.0	Processamento de imagens — histórico de CVEs de heap overflow
gunicorn	26.2.0	Servidor WSGI em produção — HTTP Request Smuggling é um vetor crítico
mysqlclient	2.2.8	Driver de banco de dados — falhas podem levar a SQL Injection

Fonte: Elaborado pelos autores.

2.3 GITHUB CODEQL — SAST (STATIC APPLICATION SECURITY TESTING)

O CodeQL é a ferramenta nativa de análise semântica do GitHub. Diferente de ferramentas baseadas em padrões de texto, o CodeQL constrói um modelo do código-fonte (grafo de fluxo

de dados) e detecta vulnerabilidades complexas que só se manifestam ao longo de múltiplos arquivos e chamadas de função.

Aspecto	Detalhe
Desenvolvedor	GitHub (Microsoft)
Linguagem analisada	Python
Query suite	security-and-quality
Resultado	Alertas publicados na aba Security → Code scanning do repositório
Permissão necessária	security-events: write
Disponibilidade	Gratuito para repositórios públicos — opção escolhida por ser repositório público

Fonte: Elaborado pelos autores.

Vulnerabilidades detectáveis no contexto Django:

- a) SQL Injection via queries brutas (`raw()`, `extra()`);
- b) Path Traversal em upload de arquivos;
- c) Server-Side Template Injection (SSTI);
- d) Uso inseguro de funções criptográficas (ex.: MD5 para dados sensíveis);
- e) Desserialização insegura.

Por que CodeQL e não Semgrep? A instrução da atividade permite explicitamente o uso do CodeQL como opção bônus para repositórios públicos, aproveitando a análise semântica nativa do GitHub. O CodeQL oferece detecção mais profunda de fluxos de dados maliciosos do que o Semgrep open-source.

3 INTEGRAÇÃO COM O DOCUMENTO DE REQUISITOS

O documento Health-In-Sight-Documentacao-Tecnico-Cientifica.pdf estabelece requisitos de segurança que a esteira DevSecOps auxilia a monitorar e garantir continuamente:

Requisito do Projeto	Ferramenta na Esteira	Como protege
Chave AES-256 para cifrar segredos do 2FA em repouso (FIELD_ENCRYPTION_KEY)	TruffleHog	Detecta se a chave for commitada acidentalmente no histórico Git
Django Secret Key nunca exposta (DJANGO_SECRET_KEY)	TruffleHog	Varre todos os commits em busca de padrões de chave Django
Credenciais do banco MySQL protegidas (DB_PASSWORD)	TruffleHog	Identifica strings de conexão e senhas expostas
Dependências atualizadas e livres de CVEs conhecidos	Trivy SCA	Bloqueia merge se pacotes como Django, Pillow ou cryptography tiverem CVE HIGH/CRITICAL
Código livre de vulnerabilidades de injeção e manipulação de dados	CodeQL SAST	Analisa o fluxo de dados no código Python e detecta SQL Injection, Path Traversal, etc.
Autenticação de dois fatores (2FA) com django-otp	CodeQL SAST	Detecta bypasses lógicos na implementação do fluxo de autenticação
Bloqueio após tentativas falhas (django-axes)	CodeQL SAST	Verifica implementação correta de controles de acesso e lógica de bloqueio

Fonte: Elaborado pelos autores.

Proteção contínua: A cada novo commit, a esteira revalida todos os requisitos de segurança acima de forma automática. Qualquer regressão (ex.: remoção de um controle, adição de uma dependência vulnerável) é detectada antes de chegar à produção.

4 EVIDÊNCIAS DE FUNCIONAMENTO

4.1 ARQUIVO DE PIPELINE (devsecops-pipeline.yml)

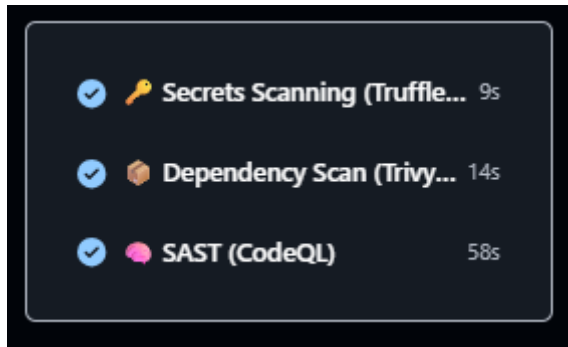
```
name: DevSecOps Pipeline
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]
jobs:
  secrets-scanning:
    name: " Secrets Scanning (TruffleHog) "
```

```

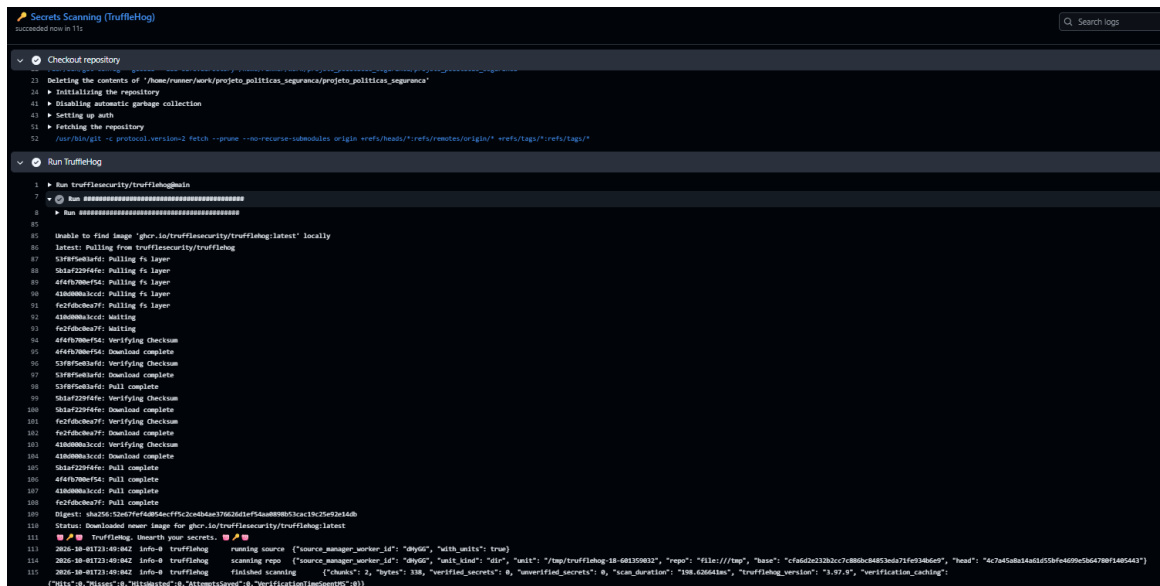
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0
  - uses: trufflesecurity/trufflehog@main
    with:
      path: ./
      extra_args: --only-verified
dependency-scan:
  name: " Dependency Scan (Trivy SCA)"
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: aquasecurity/trivy-action@master
      with:
        scan-type: fs
        scan-ref: .
        scanners: vuln
        severity: HIGH,CRITICAL
        exit-code: "1"
        ignore-unfixed: true
        format: table
sast-codeql:
  name: " SAST (CodeQL)"
  runs-on: ubuntu-latest
  permissions:
    actions: read
    contents: read
    security-events: write
  steps:
    - uses: actions/checkout@v4
    - uses: github/codeql-action/init@v3
      with:
        languages: python
        queries: security-and-quality
    - uses: github/codeql-action/autobuild@v3
    - uses: github/codeql-action/analyze@v3
      with:
        category: /language:python

```

4.2 EXECUÇÃO DA ESTEIRA — GITHUB ACTIONS



4.3 JOB: SECRETS SCANNING (TRUFFLEHOG)



4.4 JOB: DEPENDENCY SCAN (TRIVY SCA)

Cenário	Ferramenta	Comportamento do Pipeline
Segredo verificado no histórico	TruffleHog	Job falha — exit code 1 — merge bloqueado
CVE HIGH ou CRITICAL em dependência	Trivy	Job falha — exit code 1 — merge bloqueado
Vulnerabilidade no código Python	CodeQL	☐ Alerta publicado na aba Security
Nenhuma falha encontrada	Todos	Pipeline verde — merge liberado

Fonte: Elaborado pelos autores.

5 CONCLUSÃO

A esteira DevSecOps implementada eleva o nível de maturidade de segurança do projeto Health In Sight ao automatizar três vertentes complementares: detecção de segredos expostos, análise de vulnerabilidades em dependências e inspeção estática do código-fonte.

O pipeline é ativado automaticamente a cada modificação na branch principal, garantindo que nenhuma alteração passe sem revisão de segurança. A política de bloqueio assegura que falhas de severidade alta ou crítica impeçam o avanço do código até a produção, reduzindo significativamente a janela de exposição a vulnerabilidades conhecidas.

Essa abordagem alinha-se diretamente aos requisitos de confidencialidade, integridade e disponibilidade descritos no documento de segurança do projeto, reforçando controles já existentes como autenticação 2FA, hashing Argon2 e bloqueio por tentativas falhas.